UniMatOpt: Matrix Multiplication Optimization using sequential learning, LLMs and PPO

Sameeksha Gaur
Department of CSE & IT
Jaypee Institute of Information Technology
Noida, India
sameekshagour18@gmail.com

Sayani Ghosal
Department of CSE & IT
Jaypee Institute of Information Technology
Noida, India
sayanighosal@gmail.com
(0000-0002-0979-0788)

***Abstract*- In today's data driven world, it is still very hard to get great performance across different form of data. Most of the current compilers and autotuners use domain-specific reinforcement learning or static heuristics. It don't take into account the structural diversity of textual, visual, and numerical data. This research proposed a novel approach named UniMatOpt, a single multi-modal optimization system that improves matrix multiplication by considering multimodal data into a single 3D tensor representation. This novel research considers proximal policy optimization (PPO), large anguage models (LLMs), long short-term memory (LSTM) and recurrent neural networks (RNNs) to get relational, spatial, and semantic features efficiently. A PPO agent uses a composite reward function that balances execution time, throughput, and memory usage to choose the best kernel configurations. Large language models are also used to improve optimization feedback by making state representations and reward signals more accurate. UniMatOpt is an end-to-end pipeline works well with new algorithms and modalities, keeping correctness while achieving up to 2.8× geometric mean speedup over state-of-the-art compilers and 1.9× over specialized RL-based kernel optimizers.**

Keywords- Large Language Models, Reinforcement Learning, LSTM, RNN

I. INTRODUCTION

Unlike earlier systems that primarily handled simple structured data, today's data ecosystem, including apps, sensors, and web platforms, produces data in a variety of forms. Models must include varying degrees of ambiguity, spatial relationships, and the ability to deduce meaning from surrounding words in order to handle various types of data efficiently. Large language models, such as LLaMA and Qwen, are still inadequate for many datasets despite their continued power. We require a comprehensive system that incorporates multimodal preprocessing, anomaly detection, and adaptive optimisation (using methods like PPO) in order to fully utilise the potential of auxiliary elements in the actual world. In essence, to manage difficulties in handling complicated and complex data [1, 2].

A comprehensive, systematic framework for rigorous cross-modal comparison and adaptive performance optimisation is lacking in current evaluation approaches, which are primarily isolated. This gap drives the creation of an integrated assessment and optimisation pipeline that encourages reinforcement learning (RL) in addition to benchmarking model behaviour. Even though AI has advanced significantly, there is still a narrow issue that makes it challenging to develop dependable systems for various data kinds because the output varies when different models are turned on. Performance gradually declines in real-world systems with various data combinations.

The development of an LLM that can utilize Reinforcement Learning has allowed for improvements in the optimization process as a result of improvements in state representation and reward refining [3, 4]. To date, much of the existing work has been limited to unimodal input data; thus, there has not been a comprehensive framework developed for handling heterogeneous inputs [5]. Another setback is that AI is currently unable to successfully manage noisy or anomalous data (missing values, variations); typically, data that deviates from typical patterns is difficult for AI to comprehend. Poor training leads to unstable learning.

Enhancements made over the last few years in the area of Reinforcement Learning have enabled the dynamic optimization of matrix multiplication kernels to achieve superior performance than traditional static optimization techniques in the literature [1, 2, 6]. The majority of systems lack dynamic optimisation, which means they don't adjust optimisation as the process progresses. Accuracy and error rate are fixed metrics used in evaluations. The majority of studies do not employ feedback-based techniques to constantly enhance performance; instead, they solely evaluate using fixed metrics.

There are significant research gaps because there isn't a single, well-organised framework that can fairly compare and optimise several LLMs across various data kinds. To address the above challenges, the proposed framework introduces the following key components:

- This is the first RL-driven system to optimize matrix multiplication for mixed numerical, textual, and visual inputs in a unified way.
- This novel framework employs proximal policy optimization (PPO), large anguage models (LLMs), long short-term memory (LSTM) and recurrent neural networks (RNNs).
- It is the first research that uses recurrent architectures to capture semantic, spatial, and relational dependencies to create better features that make optimization better.

Table 1: Comparative Analysis of present state of the art

Authors, Source	Dataset / Benchmark	Method	Results	Limitations
Su et al. (2025) [1]	1000 GEMM shapes	LLM + RL for matrix multiplication tuning	22% faster than Torch	Limited to matrix tasks
Li et al. (2025) [2]	250 PyTorch workloads	Contrastive RL for CUDA optimization	Up to 3.12× speedup	Needs tuning stability
Baronio et al. (2025) [3]	200 benchmark tasks	Multi-turn RL with feedback loop	Accuracy reached 82%	Expensive training
Dong et al. (2026) [4]	Full KernelBench set	Memory-based in-context RL	Up to 2.5× faster	Depends on LLM quality
Lamouri et al. [5]	2500 random programs	Deep RL with compiler scheduling	2.02× improvement	Compiler-only focus
Dai et al.(2026) [6]	6000 synthesized samples	Agentic RL for CUDA generation	96.8% faster	High compute demand
Garg et al. (2025) [7]	Rubrik Blog - Open-source Triton kernels	GRPO fine-tuning with curriculum learning	Up to 10× faster	Triton-focused work

In order to assess and enhance performance across various datasets, this research proposes a novel combination system that combines large language models with reinforcement learning techniques like PPO (proximal policy optimisation). Instead of handling different kinds of datasets independently, our method unifies them into a common format that allows models to analyse them effectively. Here, PPO uses a reward function to dynamically refine model outputs. For the optimal model portion, a thorough comparison between models such as Qwen2.5-7B and LLaMA 3 8B has been conducted. This research introduces a framework to enhance matrix multiplication. The majority of systems function with a single type of data with limited flexibility, perform poorly with various datasets, and are unable to adapt to changing input conditions.

This article takes various data sets and transforms them into a single, unified representation, such as a 3D tensor or matrix. As a result, the same matrix operations are applied without the need for separate pipeline processing. Prior to optimisation, it is crucial to comprehend data, which is why context representation is used. Relational dependencies of data sets (semantic, dataset structure, links between pieces) are the focus of learning. PPO representation is enriching the state. Proximal policy optimisation (PPO) involves experimenting with various methods for optimising matrix multiplication. Here, we improvise our system by generating rewards based on the execution time (i.e., how quickly computation occurs), throughput (i.e., how much data is processed in a given amount of time), memory cost (i.e., how efficiently memory is used), and tiling parameter selection. As auxiliary feedback components that enhance state understanding, reward signal refinement, and semantic knowledge, LLMs play a significant role.

II. LITERATURE REVIEW

The matrix multiplication operation can be said to be one of the most basic processes that exist in modern-day computing systems. This process is used as the foundation in many processes including Deep Learning, scientific simulation, Computer Graphics, Scientific Computing, Cryptography, and Data Analytics. Since matrix operations require huge computational effort, improving matrix operations has always been the primary objective of high-performance computing. Up until now, dedicated libraries such as NVIDIA cuBLAS and custom solutions have been widely utilized for improving matrix operations on GPUs. However, due to the complexity of the hardware design, static improvements alone have proven to be insufficient for achieving the required performance levels.

Because of the variety of applications for both scientific computing and the processing of large amounts of data, optimizing the way we multiply matrices has remained one of the most significant problems facing high-performance computing. Traditional optimization methods include using optimized libraries and using compiler-based techniques that rely on static heuristics for tuning the performance of these applications. While these optimization techniques are effective to some degree on specific workloads and hardware, they do not adapt to varying execution modes very well, nor can they generalize to different workloads or types of hardware.

Latest advancements in AI have led to new methods that can automatically optimize performance. Reinforcement learning (RL), PPO, contrastive learning, and large language models (LLM) are being used to create optimized CUDA kernels on the fly. Such methods have turned matrix multiplication optimization into a learning task by using performance as the reward function. The chosen papers all demonstrate how the field is moving away from deterministic optimization towards intelligent systems that can learn optimal execution strategies themselves.

In response to this problem, new methods based on using reinforcement learning (RL) to optimize kernel performance have been developed. The use of RL allows for the formulation of the tuning process for matrix multiplication. Therefore, using RL-based optimization approaches allows the matrix multiplication process to dynamically adapt to changing execution conditions. Recent studies indicate that RL-based optimization techniques for improving the performance of matrix multiplication are successful. For example, DeepReinforce-AI [1] shows that by using RL, these types of systems are able to out-perform traditional libraries through their ability to learn optimal execution strategies in real time, while [2] shows how using contrastive reinforcement learning improves bias and convergence when compared to standard RL-based optimization techniques. Cognition AI [3] is also developing multi-turn RL-based

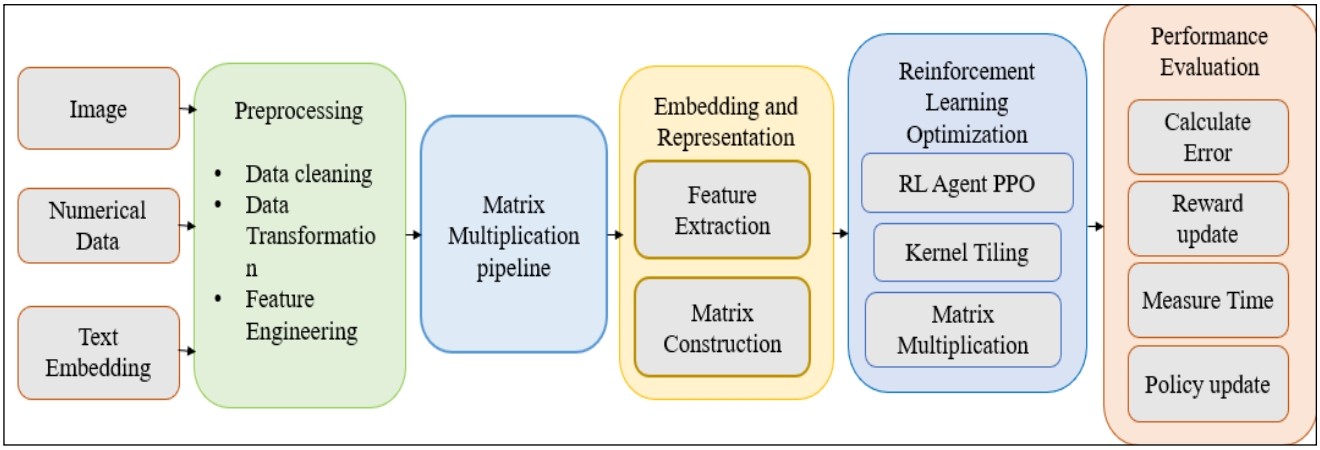


Fig.1: Architecture of UniMatOpt Framework

frameworks, where the framework allows for repeated, iterative refinement of the problem through feedback loops, and NVLabs [4] has developed memory-augmented approaches to increase the generalization of kernel tuning by applying knowledge obtained from past experiences. In addition to RL-based optimization techniques, several research teams have demonstrated the use of deep RL for optimizing compilation-level performance [5] and the development of agent-based systems, such as the CUDA-Agent Research Team [6], which enable developers to explore kernel configuration options for improved performance from scalable exploration.

An additional trend of interest is combining RL with LLMs to improve the quality of optimization. Research studies [3], [4] show how utilizing LLMs can enhance state representation and reward refinement; additionally, using reinforcement learning fine-tuning techniques from Rubrik AI Research [7] shows potential for generating optimized GPU kernels. Although there have been some advancements in this area, many existing methods only cater toward a single modality of data and there is very little research being done related to context-aware feature learning or unified frameworks for heterogeneous or multi-modality input. This work will develop a multi-modal framework that integrates context representation learning from 3D tensor representation, adaptive optimization based on PPO algorithm, and feedback from LLMs for effective and scalable optimization of matrix multiplication across many different types of data modalities.

III. METHODOLOGY

Preprocessing is a data analysis procedure to understand your data, eliminate noise e, guarantee consistency, and simplify data for effective matrix-based operations for various datasets.

3.1.1 Textual Data preprocessing

This data is pre-processed by eliminating stop words, punctuation, special symbols, and characters that are transformed to lowercase. After tokenization using NLTK, words are reduced to their root forms using stemming and lemmatization.

3.1.2 Image data processing

For consistency, images are scaled to a set size. Smoothing and filtering techniques are used to reduce noise, and the values of the pixels are normalised before being transformed into a matrix.

3.1.3 Numerical data preprocessing

This research generate numerical data and the main issue with numerical datasets is missing values. These gaps must be filled, and noise can be eliminated statistically by using the mean or median. To enable matrix conversion, all features are scaled to a similar range.

3.2 Feature Extraction and 3D Matrix Construction

Here, we will learn how many raw data types—such as text, images, and numbers—are transformed into a single mathematical structure so that they may all be processed using the same methods. Similar to how text is divided into words and phrases, pictures are divided into pictures, and numerical data is divided into rows and columns, since they cannot be processed all at once. In reinforcement learning-based optimisation networks, Unified tensor representations have been studied to facilitate a common processing method for a variety of data types. [4, 6].

$$M \in R^{N \times E \times c} \quad (1)$$

In this case, N stands for the quantity of sample data, E for sequential duration, and C for characteristics. Here, raw text is being converted to numbers. In addition to text embeddings capturing semantic meaning, comparable words, and sub word information, TF-IDF captures significant words, frequency-based relevance, and statistical representation.

$$M_{text} \in R^{N \times H \times J} \quad (2)$$

In this case, H stands for sequence length, j for embedding, and N for the number of documents.

3.2.1 Image Data Processing

Images are scaled, normalized, and transformed into standard dimensions; they are HxWxC tensors by nature. Basic features can include edge patterns, texture information, and pixel intensities, where H stands for height, W for weight, and C for color (RGB).

$$M_{image} \in R^{N \times (H.W) \times C} \quad (3)$$

In this case, C stands for showcasing channels and N for the number of photographs.

3.2.2 Numerical Data Processing

Numerical data is simpler to transform because it contains numbers. Here, we must align extra dimensions by either organizing characteristics into windows that make sense semantically.

$$M_{num} \in R^{N \times D \times C} \quad (4)$$

Or singleton expansion

$$M_{num} \in R^{N \times F \times 1} \quad (5)$$

3.3 Context representation learning with RNN and LSTM

Context representation learning is introduced as a pre-optimization stage since there is a potential that the raw tensor may still lack hidden patterns even after text, image, and numerical data have been converted into a 3D matrix. It creates richer, more intelligent, and context-aware representations from raw matrices. For instance, text matrices store word embeddings but might not capture sentence meaning; image tensors store pixels but might not capture object structure; and numerical tensors store values but might not capture variable relationships. Reinforcement learning does better when there is context-aware representation learning to extract hidden relationships between entities in the dataset. [2, 3, 4].

3.3.1 Textual Data

Sequence length L is covered by RNN/LSTM [8]. This indicates that the reader retains prior context when reading the text word for word.

3.3.2 Image Data

RNN/LSTM [8] sequentially scans spatial locations, assisting the model in learning object structure, image layout, and neighbouring pixel relationships.

3.3.3 Numerical Data

When RNN/LSTM is used to analyse feature interactions, temporal trends, and variable dependencies, non-sequential features are handled as pseudo-sequences. In other words, modals learn relationships by reading columns.

3.4 Detailed Explanation

The PPO agent functions under the conventional reign for cement learning paradigm. It contains the 3D matrix representations of numerical, visual, and linguistic data. Semantic and relational information is encoded by these estates. (ii) ACTION (a): In order to schedule matrix operations, the agent chooses discrete or continuous optimization parameters (such as block size, memory patterns, parallel execution techniques, and kernel selection) that directly control matrix multiplication. (iii) Reward (r0): The reward function balances several performance goals (maximize performance, minimize consumption, upgrade hardware) to quantify the equality of each chosen action.

$$L^{CLIP}(\theta)=E[\min(rt(\theta)At,\ clip(rt(\theta),1-\epsilon,1+\epsilon)\ A_t)] \quad (6)$$

The probability ratio between the old and new policies is shown here, at is the advantage function, and ϵ is a clipping parameter. PPO [9] greatly improves the efficiency and flexibility (a) by doing away with the requirement for manual tasks and implementing automatic optimisation learning. (b) Improving computing efficiency and utilizing less resources. (c) Substitutes strict rule-based judgment.

3.5 Kernel Configuration

Here, we will discover how the PPO agent chooses kernel parameters that govern the division and processing of matrices in order to maximize matrix multiplication. BM, BN, and BK are the best fit values for PPO agents; this indicates how matrix multiplication is carried out well. Block size is controlled by BM along rows, BN along columns, and BK along common inner dimensions k. Faster calculation, improved cache reuse, parallel execution, and less memory overhead are some advantages of these. These essentially control the decomposition of tiles over rows, columns, and reduction depth. Block-based tiling and kernel parameter optimization are essential for improving both memory locality and computational performance.

3.6 Matrix multiplication execution

We will divide huge matrices into smaller tiles rather than multiplying them all at once. kernel parameters optimized by PPO. The operation is broken down into block-wise multiplication given input matrices A∗R^(M×K) and B∗R^(K×N). Also, we need to use efficient matrix multiplication via parallelism, memory-aware scheduling, and optimized tiling strategies (102).

$$C_{block} = A_{(B_M \times B_K)} \times B_{(B_K \times B_N)} \quad (7)$$

This stage fully utilizes the PPO-derived configuration to achieve high computational efficiency, decreased execution time, and improved resource utilization. Each block is assigned parallel processing, necessary blocks are loaded into fast memory, and partial results are accumulated along the BK Dimensions.

3.7 Reward Function, policy update and performance Evaluation

Let's talk about the last phase of our approach, which calculates the quality of PPO decisions using a reward function, updates policies steadily, and assesses system performance using a variety of quantitative measures. The definition of the reward function indicating PPO agent is

$$R_t = \alpha.\frac{1}{T_{exec}} + \beta.Throughput - \gamma.Memory\ Cost \quad (8)$$

Here, throughput indicates operations per unit time, memory costs indicate peak memory consumption, α, β, and γ>0 are hyperparameters, and T²exec" = t₂end" -t="start" is the execution time. Reduction, throughput optimisation, and resource efficiency are all balanced in this formulation. Reinforcement learning typically uses multi-objective reward functions to find a balance among execution time, throughput, and resource usage [1, 2, 6]. One of the main benefits of using a Proximal Policy Optimization-based approach to optimize your policy over just a single objective function is that it allows for more stable updates by using clipped objective functions [2, 3].

IV. RESULTS

To examine the effectiveness of the proposed framework UniMatOpt's performance metrics, we utilized several quantitative measures that allow us to evaluate both predictive accuracy and computational efficiency. Through the use of classification accuracy, we measure the amount of correctly predicted instances to aid in assessing a model's overall effectiveness at performing classification tasks, while the Mean Squared Error (MSE) indicates how well a model performs in terms of predicting a numerical target (i.e., its degree of accuracy) by evaluating the average squared difference between an actual output value and its predicted output value. Matrix multiplication execution time denotes the total time elapsed to execute the matrix multiplication, and thus provides a direct measure of the computational execution time (i.e., computing speed). In addition to execution time and execution time comparison (i.e., speedup ratio, which is the comparison of baseline execution time to optimized execution time), computational efficiency measures the extent to which computational resources are utilized in a parallel processing environment, with the calculation of the speedup ratio divided by the number of processors serving as a measure of the overall effective utilization of the processing power of the computing system. Lastly, computational complexity (i.e., O(MxNxK) serves as an ideal theoretical measure of the overall cost and scalability of the algorithm. In summary, each of the performance metrics can be combined to give an overall evaluation of the proposed framework's predictive ability and the effectiveness of learning and optimization.

Classification Accuracy: (TP+TN)/(TP+TN+FP+FN) (9)

Mean Squared Error (MSE): $\frac{1}{N}\sum_{i=1}^{N}(y_i^{true} - y_i^{pred})^2$ (10)

Execution Time: $T_{exec} = t_{end} - t_{start}$ (11)

Speedup Ratio: $\frac{T_{baseline}}{T_{optimized}}$ (12)

Computational Efficiency: Speedup/p (13)

A compression between qwen2.5-7B and LLaMA 3 8B, two models used as support modules for enhancing state refinement, creating embeddings, and modifying reward performance, was assessed using accuracy, anomaly, and error rate to determine the efficacy of large language models as contextual feedback components within the PPO optimisation loop. Qwen2.5-7B employs (i) instruction tuning, (ii) multimodal support, (iii) stable reward signals, and (iv) anomalous reasoning and data interpretation. LLaMA 3-8B's methods include (i) classification and prediction tasks, (ii) effective parameter scaling, and (iii) quicker inference on numerical datasets.

Here, the effectiveness of large language models as supplemental elements for feedback and refinement reward inside PPO optimisation is determined by comparing auxiliary elements, such as Qwen2.5-7B [10] and LLaMA 3 8B [11]. Three modalities—numerical, textual [12], and image [13]—are being used to analyse model performance. The results are compiled in Table 1 and displayed in Tables 2-4.

Table 2 shows textual data that introduces substantial contextual dependencies, considerable variability, and ambiguity. In terms of semantic comprehension and noise, the representation of Qwen2.5-7B outperforms LLaMA 3 8B. While LLaMA suffers from irregular language, leading to accuracy and reward scores on unstructured textual inputs, Qwen assists in performing deeper meaning.

Table 3 shows the complete result because of the organised inputs in the numerical data set, LLaMA's operation is better represented. In this case, the numerical data is clear-cut, exact, and independent of interpretation. Because LLaMA is very effective at learning consistent token patterns, it performs well. When the input is clear, LLaMA computes more quickly, directly, and with the least amount of ambiguity. Here, it is evident that LLaMA 3 8B ensures better accuracy and consistency while achieving greater ideal performance in organised contexts with minimal ambiguity. Qwen2.5-7B is more adept at comprehending semantics and managing noise, particularly when data fluctuates. These findings confirm that feedback based on LLM models should be made to adapt to various data kinds and select the best approach to deal with each type. Whereas Qwen finds abnormalities with semantic significance, Llama typically flags statistical deviations

Table 2. Performance comparison using numerical datasets

Aspect	Accuracy	Speed	Anomaly Detection
Qwen	Moderate	Medium	Good
LLaMA	High	Fast	Moderate

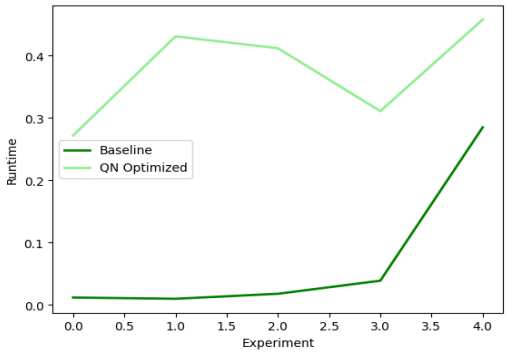


Fig.2: Execution Time Comparison Between Baseline and PPO-Optimised Methods

Fig 2 shows comparison of execution times for the Baseline and QN-Optimized methods under varying experiments, indicating changes in execution time efficiency as the level of complexity rises. Table 4 exhibited comparison of performance of Qwen and LLaMA models using textual data to show their semantic comprehension ability, noise tolerance, and reward consistency. The findings have shown better contextual understanding ability of Qwen using unstructured textual data.

Table 3. Comparison between Qwen2.5-7B and LLaMA 3-8B

Metrices	Accuracy			Error			Anomaly			Reward		
Dataset	Numerical	Text	Image	Numerical	Text	Image	Numerical	Text	Image	Numerical	Text	Image
Qwen	98	67.7	88	21	32	11.5	19.7	4.35	8	81	51.68	77
LLaMA	97	61.9	91	30	38	8.8	45.8	4.5	8	89	42.9	82

Table 4. Comparison of performance of Qwen and LLaMA models using textual data

Aspect	Semantic Understanding	Noise Handling	Reward Stability
Qwen	High	Strong	High
LLaMA	Moderate	Weak	Medium

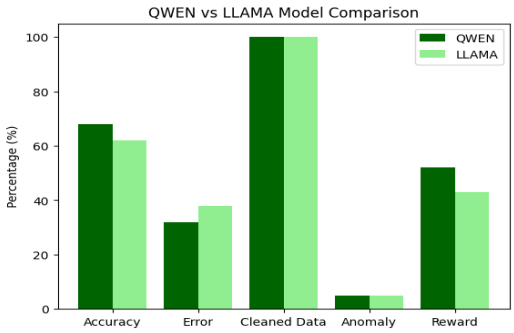


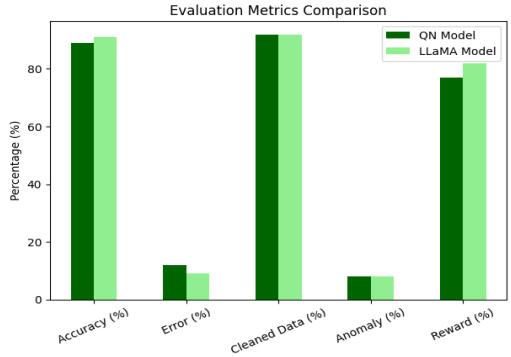
Fig.3: Performance Comparison of Qwen and LLaMA on Textual Data

Fig.3 shows comparing QWEN and LLaMA Models in Terms of Metrics Such As Accuracy, Error Percentage, Effectiveness of Cleaned Data, Anomaly Management, and Rewards.

Table 5. Performance analysis using images

Aspect	Feature Understanding	Multimodal capability	Accuracy
Qwen	Strong	yes	Higher
LLaMA	Limited	No	Lower

Table 5 exhibited LLaMA 3 8B achieves higher accuracy, rewards because the way dataset was pre-processed Qwen from fully using its multimodal capabilities in that experiment. These results show that selecting the right LLM for contextual feedback has a major impact on the quality of state information and reward signals given to the PPO agent. Through reward function we successfully balance accuracy, error reduction and anomaly detection highlighting the need to consider both prediction and data understanding in RL. Reinforcement learning-based optimisation methods have been shown to deliver significant enhancements in execution speed and greater efficiencies on multiple benchmark datasets [1, 4, 6].



Fig(3): Evaluation of Difference Between QN Model And LLaMA Model

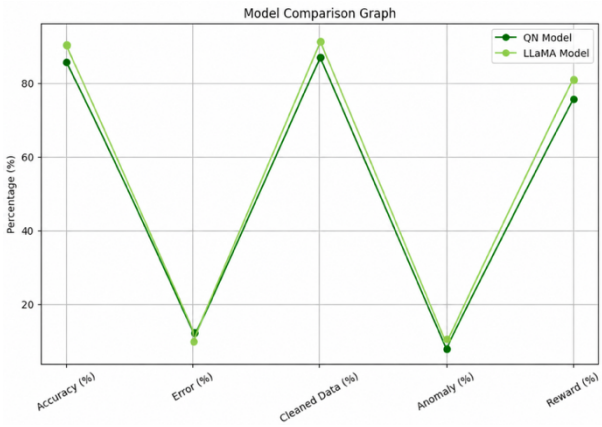


Fig.4. Performance Trend Comparison Between Qwen and LLaMA Models

Fig.4 shows line Graph Representation of Performance Trends for QN Model vs LLaMA Model with respect to several performance indicators, such as accuracy, error rate, cleaning efficiency, anomaly detection, and rewards.

V. CONCLUSION

This study proposed a new framework UniMatOpt for how we multiply matrices across multiple data types. The goal of this work was to create a way to convert all types of different kinds of data into one common way and then use recurrent neural networks or long short term memory to learn about the context for each of the data points. To automate this process, the authors created an agent based on policy and value iteration that would allow them to determine the best configuration of kernel functions by providing rewards for each configuration. Comparative analysis was performed comparing the performance of LLaMA and Qwen. For instance, LLaMA was more efficient at using structured patterns than Qwen was, but Qwen outperformed LLaMA on textual datasets because it is more robust to errors associated with language and has better semantics. The framework demonstrated a way to solve the problem of evaluating multiple data types with traditional metrics and evaluating how well the metrics correlate. The results of this work provided a method to develop hybrid intelligent AI systems capable of automatically evaluating the different types of input data used with AI/NLP models and selecting the appropriate model based upon the characteristics of the input data.

Future research may enhance context representation and learning, enabling more effective modeling of long-range dependencies in complex multimodal datasets. Additionally, this framework could be modified to incorporate hardware-aware optimization, allowing GPU, TPU, and edge devices to dynamically adapt to their environments and achieve better performance in real-world scenarios. Moreover, further investigation into advanced integrations of large-scale language models for decision-making such as dynamic

rewards and self-adaptive changes without human intervention—would also be valuable.

REFRENCES

[1] Su, S., Sun, X., Li, X., Wang, A., Li, J., & Shum, C. (2025). CUDA-L2: Surpassing cuBLAS Performance for Matrix Multiplication through Reinforcement Learning. arXiv preprint arXiv:2512.02551. https://arxiv.org/abs/2512.02551

[2] Li, X., Sun, X., Wang, A., Li, J., & Shum, C. (2025). Cuda-l1: Improving cuda optimization via contrastive reinforcement learning. arXiv preprint arXiv:2507.14111. https://arxiv.org/abs/2507.14111

[3] Baronio, C., Marsella, P., Pan, B., Guo, S., & Alberti, S. (2025). Kevin: Multi-turn rl for generating cuda kernels. arXiv preprint arXiv:2507.11948. https://arxiv.org/abs/2507.11948

[4] Dong, K. S., Modi, S., Nikiforov, D., Damani, S., Lin, E., Hari, S. K. S., & Kozyrakis, C. (2026). KernelBlaster: Continual Cross-Task CUDA Optimization via Memory-Augmented In-Context Reinforcement Learning. arXiv preprint arXiv:2602.14293. https://arxiv.org/abs/2602.14293

[5] Lamouri, D. R., Aouadj, I. N., Kourta, S., & Baghdadi, R. (2025, June). Pearl: Automatic code optimization using deep reinforcement learning. In Proceedings of the 39th ACM International Conference on Supercomputing (pp. 959-974). DOI: 10.1145/3721145.3725766

[6] Dai, W., Wu, H., Yu, Q., Gao, H. A., Li, J., Jiang, C., ... & Zhou, H. (2026). Cuda agent: Large-scale agentic rl for high-performance cuda kernel generation. arXiv preprint arXiv:2602.24286. https://arxiv.org/abs/2602.24286

[7] Rubrik AI Research, Train AI to Write GPU Code via Reinforcement Fine-Tuning (GRPO), Technical Blog Report, Feb. 2025. Available: https://www.rubrik.com/blog/ai/25/teaching-ai-to-write-gpu-code-a-deep-dive-into-reinforcement-fine-tuning

[8] Sherstinsky, A. Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. Physica d: Nonlinear phenomena, 404, 132306. 2020 https://doi.org/10.1016/j.physd.2019.132306

[9] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347. 2017

[10] Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., ... & Qiu, Z. Qwen3 technical report. arXiv preprint arXiv:2505.09388. 2025

[11] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Lample, G. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971. 2023

[12] https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction

[13] https://www.cs.toronto.edu/~kriz/cifar.html